

Auditoría técnica: Captación y organización territorial de RUANA

Fecha: 5 de agosto de 2026 Alcance: Análisis estático del repositorio completo (sin modificaciones de código)
Arquitectura: Flask (RUANA/web/app.py) + JavaScript vanilla + RUANA/core/db_manager.py (SQLite local / PostgreSQL Supabase en producción)

1. Resumen ejecutivo

RUANA dispone de un MVP de captación territorial operativo en producción. La lógica de negocio está concentrada en db_manager.py (~13.000 líneas) y se expone vía app.py. El territorio se organiza por código postal → grupos (máx. 5) → plazas por oficio principal → lista de espera (en_espera) → competencia por score.

No existen: geolocalización, radio de actuación, densidad como métrica, matching geográfico avanzado, Edge Functions de Supabase, frontend React/TypeScript, incorporación automática de suplentes.

Brechas principales: desalineación migraciones Supabase vs runtime Python; referidos-module.js implementado pero no cableado en HTML; E2E desactualizado; plazas cerradas por admin no bloquean asignación automática.

2. Panorama arquitectónico

Capa	Tecnología	Rol territorial/captación
Frontend	HTML/JS/CSS en RUANA/web/	Registro, invitación, panel aliado/admin
API	Flask app.py	Endpoints REST de registro, invitaciones, grupos, suplentes, competencia
Lógica de negocio	db_manager.py	Fuente de verdad: asignación, plazas, score, competencia
Motor evaluación	engines/motor_evaluacion.py	Semáforo evaluación (no asigna grupos)
Orquestador	core/orquestador.py	Stub heredado; no gobierna territorio en producción
BD local	SQLite ruana.db	Desarrollo/tests
BD producción	Supabase PostgreSQL	Esquema en supabase/migrations/
Config	oficios_ruana.json, ruana_reglas_v1.json	Catálogo oficios y umbrales competencia

Constantes territoriales clave (db_manager.py)

- MAX_GRUPOS_POR_CP = 5 — máximo grupos activos por código postal
- ESTADOS_GRUPO = ('activo', 'en_competencia', 'disuelto')
- RUANA_CODIGO_INVITACION_REGEX — formato RUANA-{grupo_id}-{OFICIO}-{4chars}
- Plaza: un oficio principal por grupo (especializaciones ignoradas)

3. Hallazgos detallados por componente

3.1 MAX_GRUPOS_POR_CP

Campo	Detalle
Archivo	/workspace/RUANA/core/db_manager.py línea 31
Qué hace	Fija máximo 5 grupos activos por CP
Cómo funciona	Usada en crear_aliado, incorporar_aliado_espera, competencia
Callers	db_manager.py, tests, documentación
Impacto	Crítico — densidad territorial
Uso	Activo, MVP
Riesgos	Hardcodeado, no configurable por entorno
Recomendaciones	Externalizar en ruana_reglas_v1.json

3.2 crear_aliado — algoritmo de asignación

Campo	Detalle
Archivo	/workspace/RUANA/core/db_manager.py líneas 2269-2411
Qué hace	Crea aliado y asigna grupo o estado en_espera/pendiente_validacion
Decisiones	1) Oficio fuera catálogo → pendiente_validacion. 2) Grupo invitador si plaza libre. 3) buscar_grupo_sin_oficio. 4) Crear grupo si <5. 5) Si CP lleno → en_espera.

Callers	POST /api/aliados/registrar en app.py
Impacto	Crítico
Uso	Activo, MVP
Riesgos	Lógica duplicada con completar_aliado_pendiente
Recomendaciones	Extraer motor de asignación único

3.3 MENSAJE_LISTA_ESPERA

Campo	Detalle
Archivo	db_manager.py líneas 2258-2267
Qué hace	Texto UX para aliados en lista de Suplentes
Callers	crear_aliado, register.html
Riesgos	Promete notificación; incorporación es manual por admin
Recomendaciones	Alinear copy o automatizar notificación

3.4 buscar_grupo_sin_oficio

Campo	Detalle
Archivo	db_manager.py líneas 1795-1817
Qué hace	Primer grupo activo del CP sin ese oficio ocupado
Exclusividad	Un oficio principal activo por grupo
Riesgos	No considera grupo_oficio_cerrado
Recomendaciones	Integrar plazas cerradas en búsqueda

3.5 crear_grupo_en_cp / fusión de grupos

Campo	Detalle
Archivos	db_manager.py 1867-1945, 2081-2252
Qué hace	Creación automática grupos; procesar_viabilidad_grupo fusiona si <2 aliados
Uso	Activo, MVP

3.6 Estado en_espera (lista de Suplentes)

Campo	Detalle
Archivos	db_manager.py, app.py (login 403), register.html, admin.html
Qué hace	Registrado sin plaza; sin acceso al panel
Admin	GET /api/admin/suplentes-espera, POST ../incorporar
Uso	Activo, MVP
Riesgos	Sin cola automática al liberar plaza

3.7 _buscar_retador — matching competencia

Campo	Detalle
Archivo	db_manager.py líneas 3956-4005
Prioridad	1) en_espera mismo CP/oficio (FIFO). 2) Activo en otro grupo CP, menor densidad
Uso	Activo, MVP

3.8 Competencia automática (score < 15)

Campo	Detalle
Archivos	db_manager.py 3427-4310, ruana_reglas_v1.json
Reglas	Umbral 15, 30 días, reinicio score 50, 2ª derrota → expulsado
Trigger	También en cada GET /api/aliado/datos
Uso	Activo, MVP
Riesgos	Side-effects en lectura; forzar_competencia incompleto vs flujo real

3.9 Invitaciones

Tipo	Tabla/API	Score	Uso
Simple (5 dígitos)	invitaciones	+3 invitador	MVP activo
Por oficio RUANA-*	invitaciones_oficio	+5	MVP activo
Campaña admin	invitacion_campanas	Según linaje	MVP activo
Placeholder legacy	aliados pendiente_completar	Variable	Compatibilidad

APIs: POST /api/invitaciones/crear, POST /api/generar-invitation, GET /api/validar-invitation

3.10 Referidos y linaje

Campo	Detalle
Tablas	referidos, aliados.invitado_por_codigo, invitado_origen
Lógica	asignar_invitado_por, árbol referidos, Regla 3 score
API	/api/aliado/referidos, /api/admin/referidos/*
Frontend	referidos-module.js HUÉRFANO (no cargado en HTML)
Uso	Backend activo; árbol avanzado no expuesto

3.11 Flujo «Conozco a alguien»

Campo	Detalle
Estado	candidato_pendiente en solicitudes
Funciones	marcar_solicitud_candidato_pendiente, vincular_solicitud_a_aliado_incorporado
Uso	Activo, MVP
Gap Supabase	Columnas candidato solo en runtime, no en migraciones

3.12 POST /api/aliados/registrar

Campo	Detalle
Archivo	app.py líneas 2138-2326
Qué hace	Registro público; orquesta validación, asignación, consumo invitación
Uso	Crítico, MVP

3.13 sugerir_cp_adyacente

Campo	Detalle
Archivo	db_manager.py 1891-1911
Estado	MUERTO — sin callers en código activo
Nota	Docs archivados mencionaban redirect_to_codigo_postal; código actual usa en_espera

4. Esquema SQL (Supabase)

Tablas principales

Tabla	Propósito
grupos	Unidad territorial por CP
aliados	Profesionales + estado + grupo + score
invitaciones	Códigos simples un uso
invitaciones_oficio	Códigos RUANA-oficio
referidos	Linaje invitación
competencia	Titular vs retador
grupo_oficio_cerrado	Plazas cerradas admin
solicitudes	Demanda intra-grupo
score_movimientos	Auditoría score
aliados_eliminados	Archivo post-eliminación

Funciones SQL

- set_actualizado_en() — triggers timestamps
- current_aliado_codigo() — RLS
- is_ruana_admin() — RLS

Migraciones relevantes

Archivo	Cambio
20260519000100_init_ruana_clean.sql	Esquema base
20260722000100	Linaje invitado_por_codigo
20260727000100	Plaza por oficio; retador; duplicados → en_espera
20260729000100	Score max 500
20260730000100	Email/tel únicos parciales
20260731000100	aliados_eliminados

Brecha Supabase vs runtime

Objetos activos en Python pero ausentes en migraciones Supabase:

- invitacion_campanas + usos
- competencia_pendiente
- referidos.origen (backfill migración 220 roto sin columna)
- invitaciones.solicitud_id
- solicitudes.candidato_*, asignada_a_*

No existen: vistas SQL territoriales, ENUMs nativos, Edge Functions.

5. Sistema de Score

Aspecto	Valor
Campo	aliados.score
Rango código	0-500 (README dice 0-100 — desalineado)
Límite diario	±10 puntos
Inicial registro	50
Umbral competencia	15 (ruana_reglas_v1.json)
Estados	ÉLITE 350+, DESTACADO 200+, ESTABLE 50+, EN RIESGO 15+, COMPETENCIA <15

Recompensas captación: +3 invitación simple (Regla 1); +5 invitación oficio (Regla 9); +1 padre/abuelo en encargos (Regla 3).

Riesgo: Doble modelo score — operativo (aliados.score) vs motor evaluación (evaluaciones).

6. Frontend y documentación

Funnel MVP operativo

index.html / invite.html → validar-invitacion → register.html → POST /api/aliados/registrar → aliado.html

Componentes clave

Componente	Archivo	Estado
Acceso dual	index.html	Producción, MVP
Gate invitación	invite.html	Producción, MVP
Registro	register.html	Producción, MVP
Panel aliado	aliado.html	Invitar, Conozco a alguien, oficios faltantes
Panel admin	admin.html	Campañas, suplentes, pendientes, linaje drawer
Árbol referidos	referidos-module.js	Huérfano (~725 líneas, APIs activas)
Dashboard	dashboard.js	Legacy, mock corrupto

Matriz tipos invitación (UI)

Tipo	Generador UI	Endpoint
Simple	Perfil / Conozco a alguien	POST /api/invitaciones/crear
Por oficio	Oficios faltantes grupo	POST /api/generar-invitacion
Campaña	Admin + QR	POST /api/admin/invitacion-campanas

Matriz estados post-registro

Estado	Acceso panel	Acción admin
activo	Sí	—
pendiente_validacion	No (403)	Activar/Rechazar
en_espera	No (403)	Incorporar suplente
expulsado	No	Nueva invitación

Documentación autoritativa

- /workspace/README.md §5.4-5.7 — Manual Maestro
- /workspace/docs/flujos/registro-aliados.md — Algoritmo registro
- Archive obsoleto: plaza por especialización

7. MAPA DEL SISTEMA

Flujo desde registro hasta pertenencia a grupo

1. Profesional llega a / o /invite con código.
2. GET /api/validar-invitation — tipos: RUANA-oficio, campaña, simple, placeholder.
3. register.html — captura datos + CP + oficio catálogo.
4. POST /api/aliados/registrar — genera código personal 5 dígitos.
5. Si oficio fuera catálogo → pendiente_validacion (sin grupo).
6. Si oficio en catálogo:
 - Grupo invitador con plaza → asignar.
 - Otro grupo CP sin oficio → asignar.
 - Sin plaza y <5 grupos → crear grupo.
 - 5 grupos y oficio ocupado → en_espera.
1. Consumir invitación → referidos + score + linaje.
2. Si activo → login panel; si en_espera → sin acceso; si pendiente_validacion → admin activa.
3. Score < 15 → competencia (retador preferente de en_espera).
4. Perdedor 1ª vez → score 50, nuevo grupo; 2ª vez → expulsado.

Decisiones del sistema

- Validación F07 (nombre ≥3, email, teléfono ≥7 dígitos).
- Canonización oficio vs oficios_ruana.json.
- Prioridad grupo invitador.
- Exclusividad 1 oficio/grupo.
- Máximo 5 grupos/CP.
- Sin plaza → en_espera (no rechazo).
- Competencia 30 días por mayor score.
- Viabilidad grupo <2 aliados → fusión/disolución.

8. ORGANIZACIÓN TERRITORIAL

Códigos postales

- Unidad territorial única operativa.
- Matching por CP exacto (string).
- Sin geocodificación ni radio.
- sugerir_cp_adyacente existe pero no se usa.

Grupos

- Máximo 5 activos por CP.
- Nombre autogenerado RUANA-<ID>-<SUFJO>.
- Estados: activo, en_competencia, disuelto.
- Creación automática al registrar.
- Fusión si grupo con 1 aliado.

Oficios

- Catálogo: RUANA/config/oficios_ruana.json.
- Una plaza = un oficio principal por grupo.
- Admin puede cerrar/reabrir plazas (grupo_oficio_cerrado).

Aliados

- Estados: activo, en_espera, pendiente_validacion, expulsado, etc.
- Límite efectivo por grupo = plazas de oficio en catálogo.

Límites

Límite	Valor	Implementado
Grupos/CP	5	Sí
Oficio/grupo	1 activo	Sí
Score máximo	500 (código)	Sí
Derrotas → expulsión	2	Sí

Disponibilidad y expansión

- Plaza libre = sin activo con ese oficio en grupo.
- Expansión = nuevo grupo hasta límite 5.
- Incorporación suplentes = manual admin.

Restricciones

- en_espera sin panel.
- expulsado sin login; contacto liberado.
- RLS Supabase: escritura territorial vía Flask/service role.

9. CAPTACIÓN

Ya implementado

- Registro público POST /api/aliados/registrar.
- Validación invitación (simple, oficio, campaña, legacy).
- Invitación aliado y por oficio faltante.
- Campañas admin multiusuario con QR.
- Referidos + linaje + score.
- Flujo «Conozco a alguien» con candidato pendiente.
- Email bienvenida SMTP.
- Lista Suplentes (en_espera) + incorporación admin.
- Activación oficios fuera catálogo.
- Métricas admin (ratios captación, zona demanda).

Parcialmente implementado

- Lista espera: estado sí; incorporación manual.
- Competencia pendiente: cola sin retador.
- Campañas masivas: funcional, ampliable.
- Score UI: backend 0-500; docs parcialmente 0-100.
- referidos-module.js: código listo, no en UI.

Solo documentado / obsoleto

- Redirección CP adyacente al llenar 5 grupos.
- Zonas Centro/Norte/Sur en dashboard.js (mock).
- Reinicio score 75 tras competencia (código = 50).
- Plaza por especialización (archive).

No existe

- Geolocalización, radio, densidad calculada.
- Vacantes como entidad SQL.
- Matching por proximidad.
- Expansión >5 grupos/CP.
- Edge Functions Supabase.
- Waitlist automática.

10. Tabla resumen funcionalidades

FUNCIONALIDAD	ESTADO	ARCHIVOS	OBSERVACIONES
Registro aliados	Implementado	app.py, register.html, db_manager.crear_aliado	Score inicial 50
Asignación automática grupo	Implementado	db_manager.py	Prioridad invitador → CP → crear grupo
Límite 5 grupos/CP	Implementado	MAX_GRUPOS_POR_CP	Hardcodeado
Exclusividad oficio/grupo	Implementado	_grupo_tiene_oficio	Especializaciones ignoradas
Lista espera (Suplentes)	Parcial	en_espera, admin.html	Sin auto-incorporación
Invitación simple	Implementado	invitaciones, /api/invitaciones/crear	+3 score
Invitación por oficio RUANA-*	Implementado	generar_invitacion_oficio	+5 score
Campañas admin	Implementado	invitacion_campanas	Multiuso QR
Referidos / linaje	Implementado	referidos, referidos-module.js	Módulo JS huérfano
Conozco a alguien	Implementado	marcar_solicitud_candidato_pendiente	Vincula solicitud
Competencia por score	Implementado	competencia, _buscar_retador	Umbral 15, 30 días
Expulsión 2ª derrota	Implementado	_finalizar_una_competencia	Requiere nueva invitación
Fusión/disolución grupos	Implementado	procesar_viabilidad_grupo	Mínimo 2 aliados
Cierre/reapertura plazas admin	Parcial	grupo_oficio_cerrado	No filtra asignación auto
Catálogo oficios	Implementado	oficios_ruana.json	~30 oficios
Geolocalización / radio	No existe	—	Solo CP texto
Vacantes (entidad)	No existe	—	Plaza inferida
CP adyacente sugerido	Muerto	sugerir_cp_adyacente	Sin callers
Edge Functions	No existe	—	Lógica en Flask
Paridad migraciones Supabase	Parcial	supabase/migrations/	Brecha vs runtime
E2E registro UI	Parcial	e2e/	Referencia suboficios eliminados
Árbol referidos UI	Parcial	referidos-module.js	No cableado en HTML

11. Riesgos transversales

1. Doble modelo score — operativo vs motor evaluación.
2. Plazas cerradas ignoradas en buscar_grupo_sin_oficio.
3. forzar_competencia incompleto vs _iniciar_competencia_si_procede.
4. Competencia en cada GET /api/aliado/datos — side-effects en lectura.
5. Brecha migraciones Supabase — runtime más amplio que SQL versionado.
6. referidos-module.js huérfano — ~1100 líneas sin uso UI.
7. E2E desalineado — #suboficios-section eliminado.
8. Documentación score — README 0-100 vs código 0-500.
9. db_manager.py monolítico — ~13k líneas, alto riesgo regresión.

12. Recomendaciones priorizadas

Prioridad	Acción
P0	Actualizar E2E al modelo oficio principal único
P1	Cablear o eliminar referidos-module.js
P1	Alinear migraciones Supabase con runtime; corregir referidos.origen
P1	Respetar grupo_oficio_cerrado en asignación automática
P2	Alinear forzar_competencia con flujo real
P2	Job programado para competencia y viabilidad grupos
P2	Notificar suplentes al incorporar
P3	Parametrizar MAX_GRUPOS_POR_CP en config
P3	Unificar documentación score 0-500
P3	Extraer módulos desde db_manager.py

13. Índice endpoints territoriales

Ruta	Método	Dominio
/api/aliados/registrar	POST	Registro + asignación
/api/validar-invitation	GET	Pre-validación

/api/invitaciones/crear	POST	Captación aliado
/api/generar-invitaacion	POST	Captación oficio
/api/aliado/datos	GET	Grupo, competencia
/api/aliados/directorio	GET	Mismo grupo + CP
/api/solicitudes	GET/POST	Demanda territorial
/api/catalogo/oficios	GET	Catálogo
/api/admin/suplentes-espera	GET	Lista espera
/api/admin/.../incorporar	POST	Incorporar suplente
/api/admin/forzar-competencia	POST	Competencia manual
/api/admin/cerrar-oficio	POST	Cerrar plaza
/api/admin/invitacion-campanas	GET/POST	Campañas QR
/api/purga/mensual	POST	Purga pool
/api/aliado/referidos	GET	Red captación

14. Conclusión

RUANA tiene un MVP de captación territorial funcional con funnel completo invitación → registro → asignación → panel → re-invitación. La lógica está en Python (db_manager.py); el frontend vanilla opera el flujo principal; Supabase aporta persistencia con brechas de paridad. Las mejoras prioritarias son deuda técnica (módulo referidos, E2E, migraciones) y cierre de gaps operativos (plazas cerradas, automatización suplentes), no ausencia del modelo de negocio base.

Documento generado a partir de la auditoría técnica del 5 de agosto de 2026. Sin modificaciones de código en el repositorio.